ECE 3720 Final Project

Engineers: Eric Thelen, Ethan Coss, Samuel Robin, and Tim Liu

Project Motivation

Samuel's Project Inspiration Story:

1. Long-standing interest in music and piano performance
2. Inspired by piano-learning apps that provide real-time feedback
3. Curiosity about how systems recognize musical notes from sound
4. Proposed the idea to the project group as a music-focused embedded systems application
5. Chosen because it combines:

- Analog sensing
- Microcontroller programming
- Digital signal processing
- Real-world user interaction

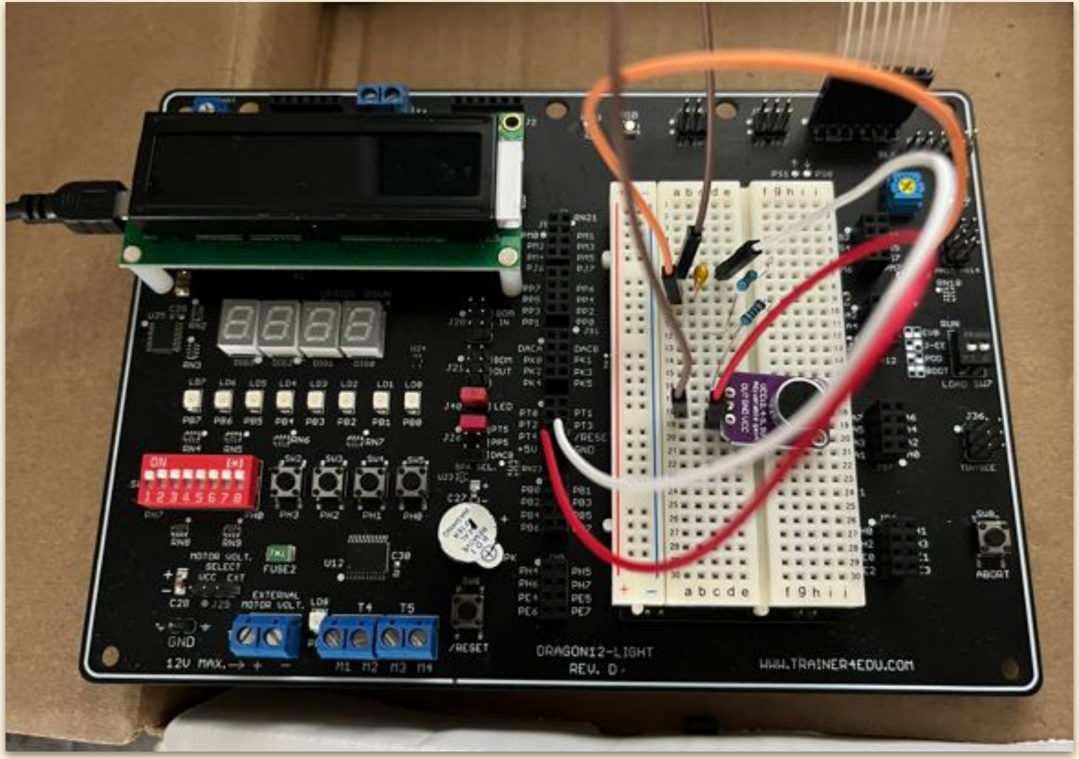
Musical Note Detector *with playback Feature*

The Project makes use of the dragon HCS12 development board, microphones, and RC filters programmed in C to identify unique, ambient sound characteristics [1][2][3].

The goal was a system to create a functions unlike a standard piano or guitar tuner. This system looks for notes and stores them for later recall and playback.

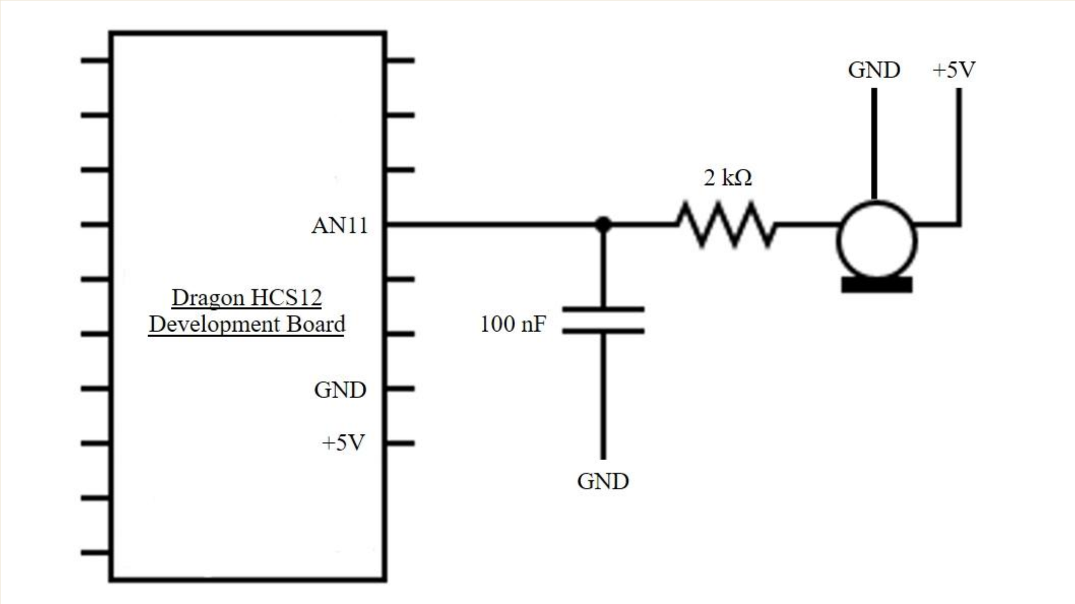
Very basic and widely known algorithms were implemented to find information based on the raw input from the microphone.

Data was stored on board in the system memory and then played back at defined intervals once the a certain amount of notes was detected.



Schematic

Image Courtesy: [2]



<u>Electret microphone</u>

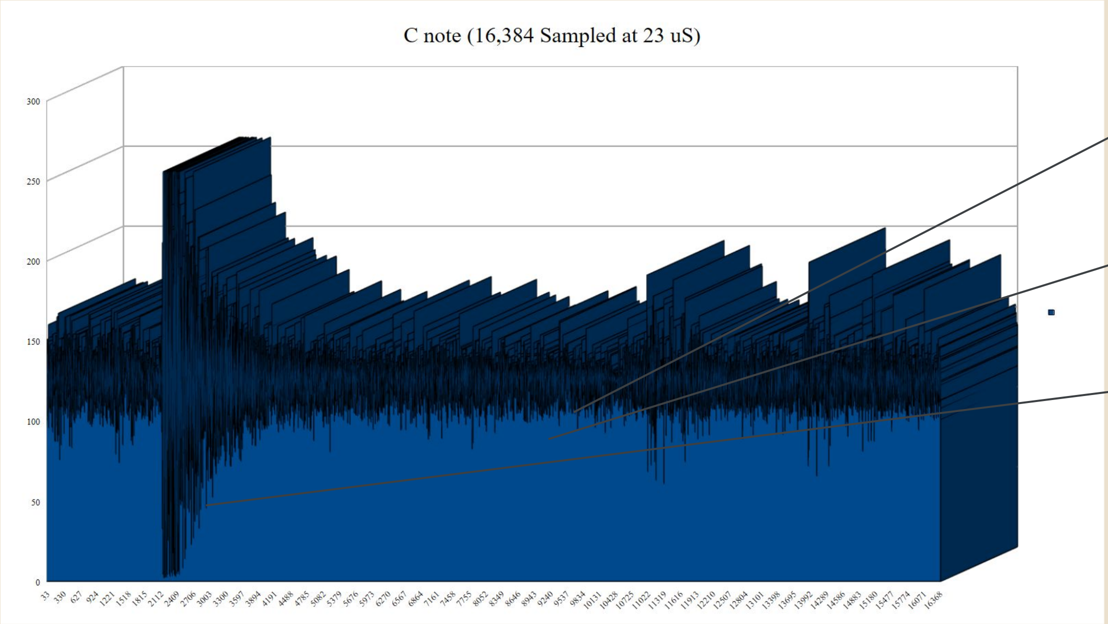
Frequency spectrum: 20Hz-20kHz
FFT capable: yes
Supply voltage: +2.4V - +5.5V [2]

<u>HCS12 board</u>: Pin AN11

Only one analog pin required [1]

<u>RC Low-pass filter</u>

Uses 2 1k ohm resistors
Uses 1 100 nanofarad capacitor
Blocks frequencies above 800Hz [3]



Common sampling rate [4]

Roughly 40kHz works for sound sampling [5]

Long dead periods

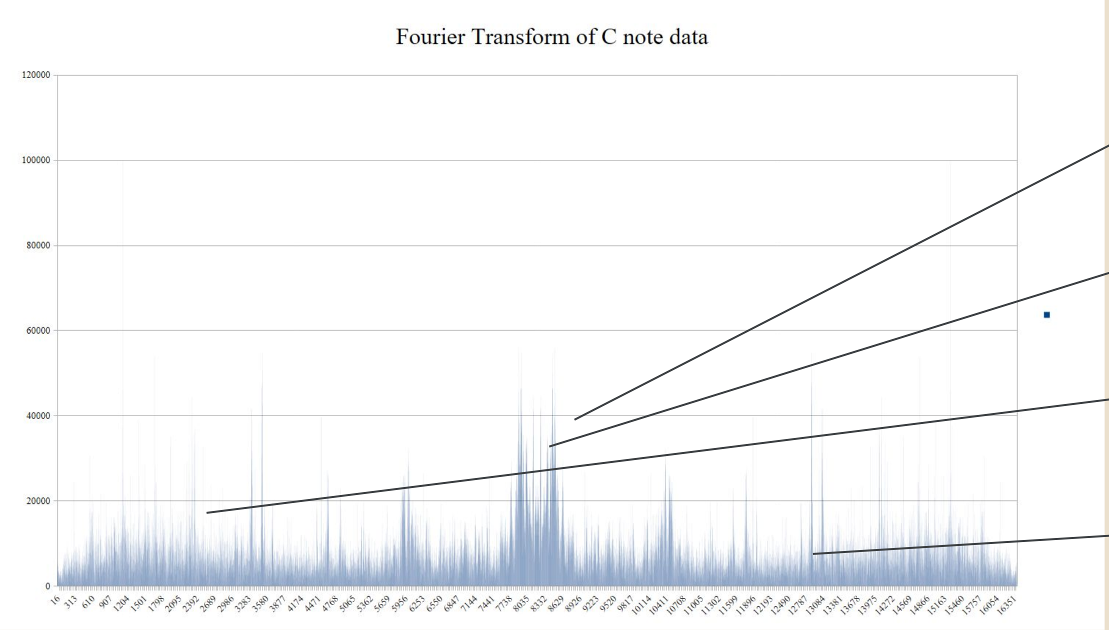
Note pattern only last a portion of the sample

ADC zero living at 128

The sample wasn't centered at zero

1

Meet the Data



No predictable pattern
What the team was getting looked like noise

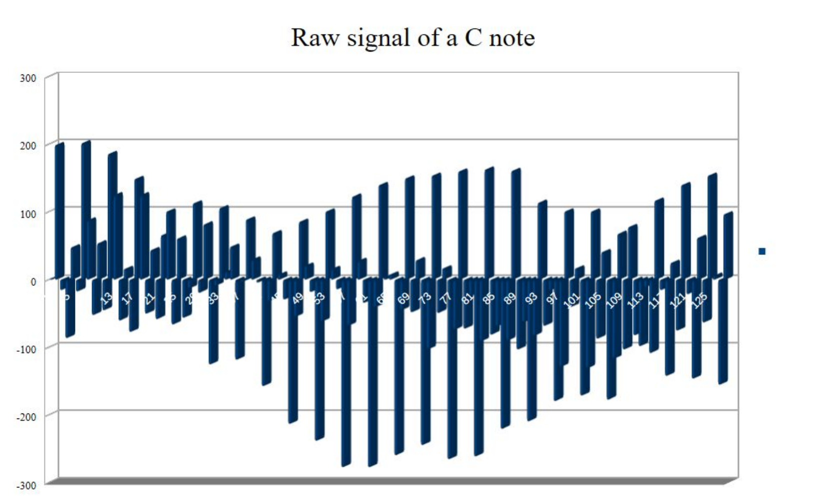
High frequency noise present
This sample was taken in lab

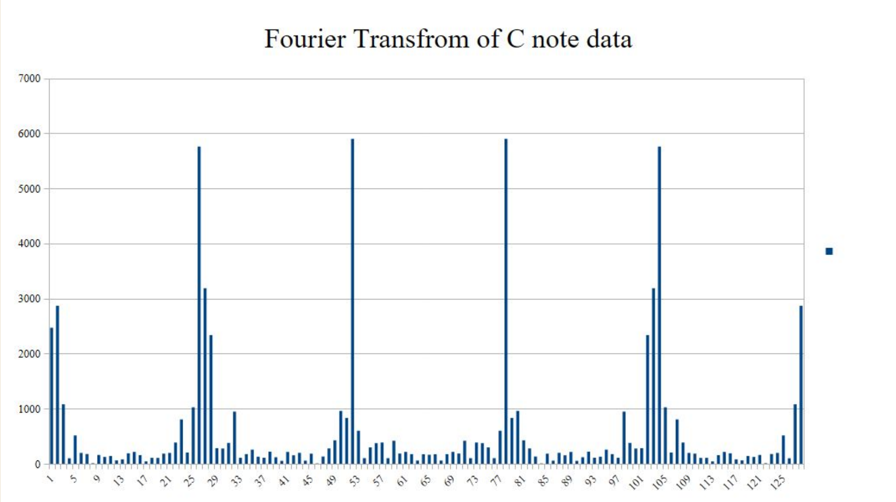
Repeatable pattern at low freq.
The notes live here [4]

Overall, too many samples
The DFT was too large [5]

2

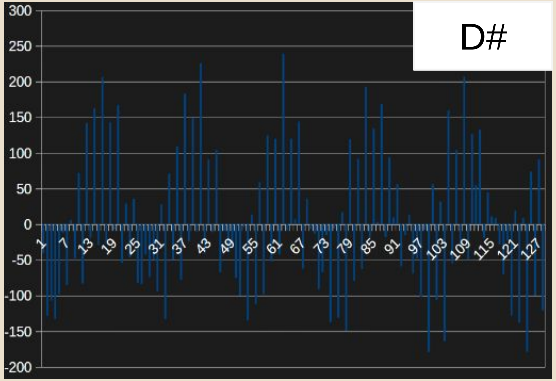
Meet the Data

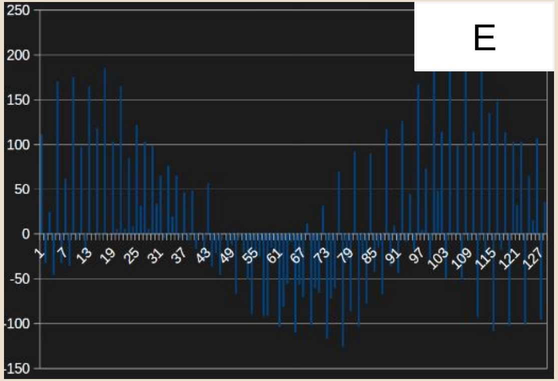




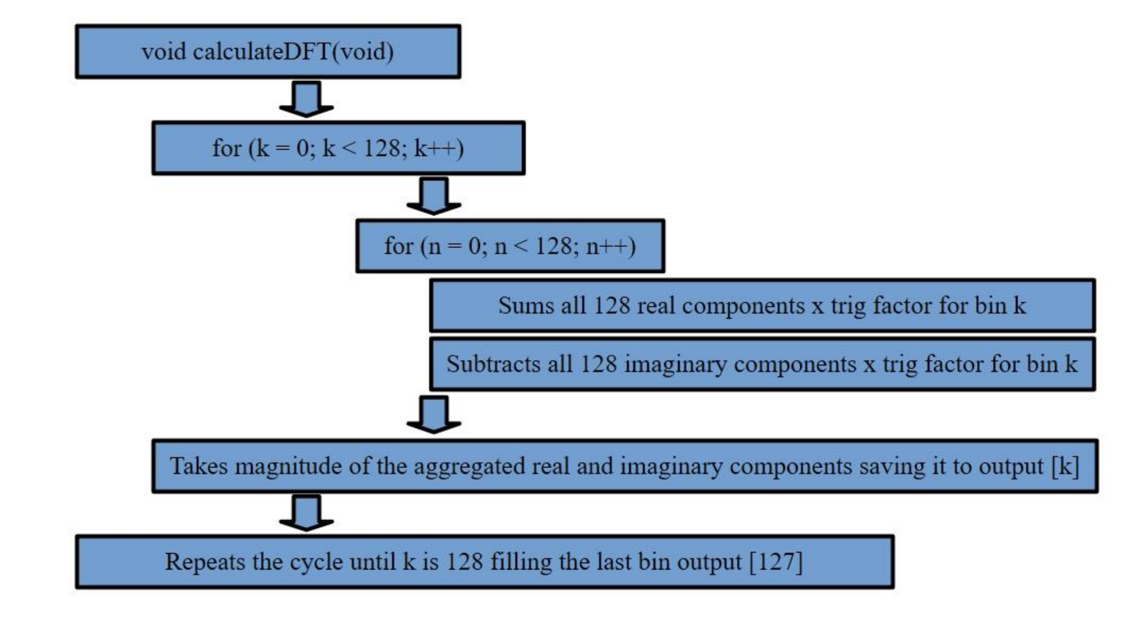
Meet the Data

3





Meet the DFT Algorithm

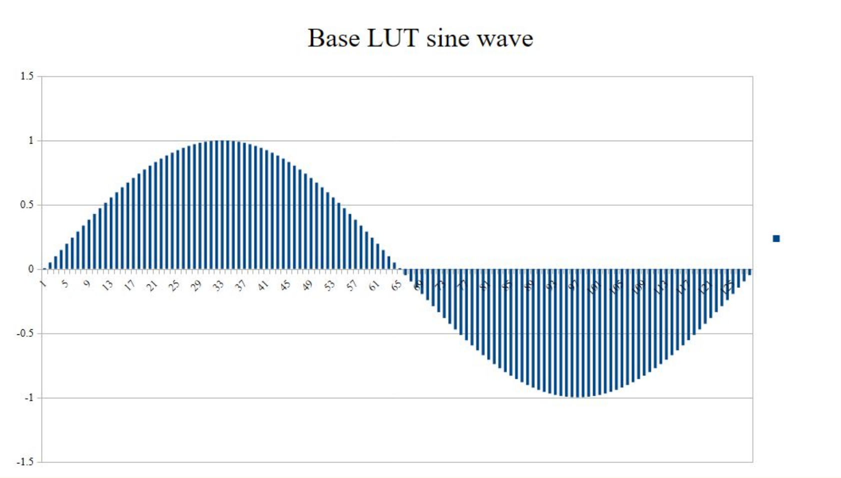


Giving a brief description, a Fourier transform works by taking a fixed size sample set and multiplying it by increasing frequency wave coefficients.

If a particular frequency aligns well with one of those found in the sample. It amplifies the bin in which it is found.

Our DFT is simply 2 for loops that index 128 samples for 128 bins [6].

Meet the DFT Algorithm



Index	Base sine LUT	Bin 4		
0	0	0	3	Entry 0
1	0.04906954681	0.14673603781	3	Entry 1
2	0.09802087176	0.29029544134	3	Entry 2
3	0.14673603781	0.4275703461	3	Entry 3
4	0.19509767689	0.55558893838	3	
5	0.24298927269	0.67157979097	3	
6	0.29029544134	0.77303186076	3	
7	0.33690220943	0.85774884935	3	
8	0.38269728861	0.92389674997	3	
9	0.4275703461	0.97004355114	3	
10	0.4714132705	0.99519023781	3	

To reduce the amount of times the Trigonometric functions were called:

A base Sine and Cosine wave were implemented for one full cycle:

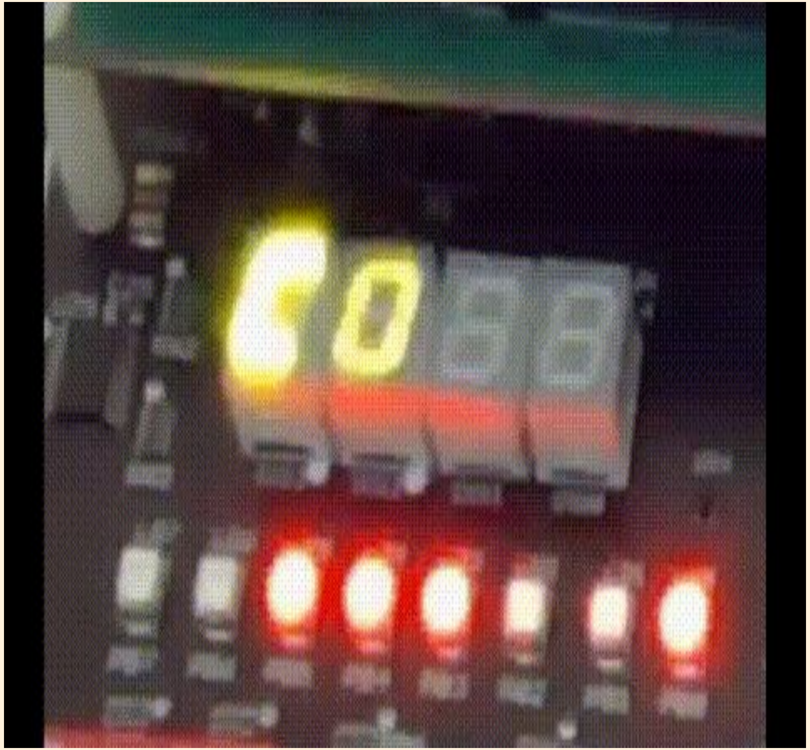
Sin(2*PI*(1/128))

Cos(2*PI*(1/128))

Because every entry that is called is an interior frequency shift in the DFT, it is possible to jump ahead in the table to call that entry rather than have to re-calculate the whole function [7].

A while loop rolls the index over to get the right entry.

Playback Function



1

Store the 5 notes found

Every key that is pressed and identified gets 2 pieces of data stored about it. The note and its sharp.

2

Detect the 5th note

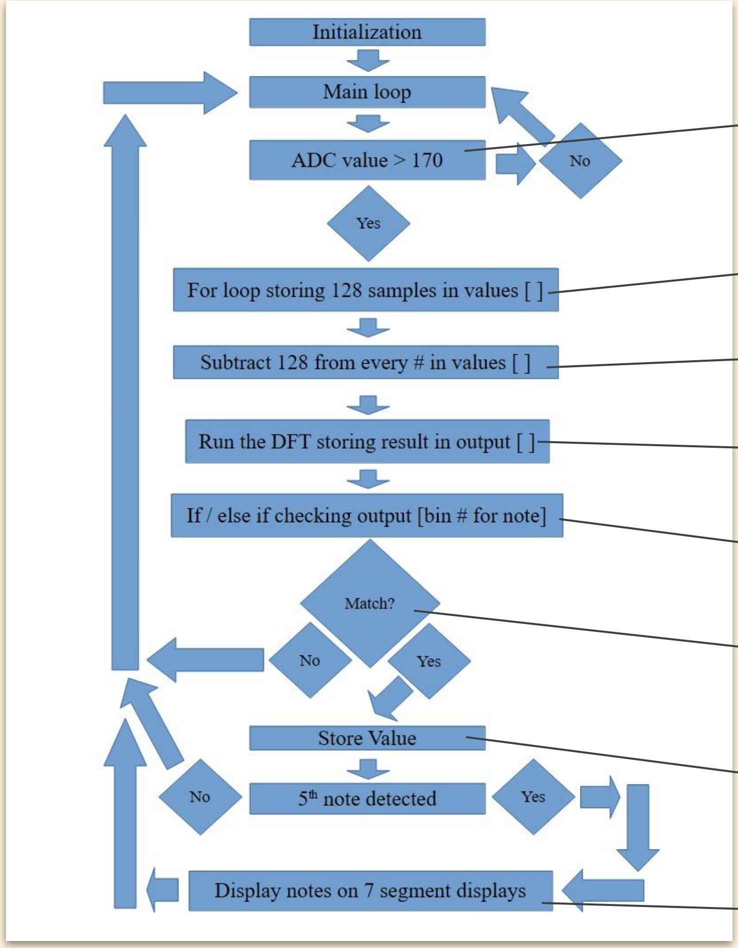
Upon filling all note spaces, the program detects completion and calls the playback.

3

Playing back the note

The note and sharp are passed to the 7 segment indexer and played at defined intervals after which the program goes back to the main loop.

Code execution diagram



Microphone breaks sound limit
A note has been played .

Pulling 128 samples
Sampling rate of our choosing.

Center on zero
Subtract 128 from every sample.

Run the DFT
The DFT is fixed size.

Bank of if / else if / else
Checking against known note profiles.

A match? Yes, move on
No, return to the start of the loop.

Storing notes
At the 5th note, jump to final playback

Playback the notes
Chase the notes and return to loop start

See it in Action

https://drive.google.com/file/d/1vO93p2WqoV-baHl06p6CysRUKkgoCScn/view?usp=drive_link

Potential Improvements and hurdles

1

Hurdle 1: Some the the notes get confused with one another in detection, specifically G and A#. This is probably because the sampling rate needs to be adjusted to differentiate some notes more closely.

This could be found with more empirical testing.

2

Improvement 1:

The note training process is long And tedious. In the real world, a system that recursively learns the note profiles from real world data on its own could be imagined.

This would require the use of higher algorithms than the team currently understands.

3

Improvement 2:

The HCS12 architecture was a very good system for implementing a basic algorithm. However, the team was already pushing the limits of what the system could handle.

An improved system might make use of a controller with higher sampling rates and more memory.

A final look

1. Target 1: Achieved

   Successfully spec out and implement hardware early

2. Target 2: Achieved

   Successfully define and implement hardware and software filters to get rid of noise

3. Target 3: achieved

   Choose an algorithm that can be implemented in the C family language without C++

4. Target 4: Achieved

   Create and achieve a physical playback feature on the HCS12 board

Playback function

Questions and Answers?

Bibliography

[1] Trainer4Edu, dragon12_light_hcs12_manual_d.pdf, 2026. [Online]. Available: https://moodle.oakland.edu/pluginfile.php/10700678/course/section/3461689/dragon12_light_hcs12_manual_d.pdf. [Accessed: Aug. 12, 2026].

[2] “6 Pieces Electret Microphone Amplifier Module MAX4466 Adjustable Gain Blue Breakout Board,” Amazon.com. [Online]. Available: https://amazon.com/dp/B08N4FNFTR. [Accessed: Aug. 12, 2026].

[3] Digikey, “Low Pass/High Pass Filter Calculator,” www.digikey.com. [Online]. Available: https://www.digikey.com/en/resources/conversion-calculators/conversion-calculator-low-pass-and-high-pass-filter. [Accessed: Aug. 12, 2026].

[4] Mark Van Raamsdonk, “Frequency spectrum of musical notes,” YouTube, Dec. 2, 2020. [Video]. Available: https://www.youtube.com/watch?v=RIkvjUQFe7s. [Accessed: Aug. 12, 2026].

[5] Mark Newman, “Why is the output of the FFT symmetrical?,” YouTube, Jun. 28, 2022. [Video]. Available: https://www.youtube.com/watch?v=IIofPiVVC64. [Accessed: Aug. 12, 2026].

[6] GeeksforGeeks, “Discrete Fourier Transform and its Inverse using C,” www.geeksforgeeks.org, Jul. 30, 2026. [Online]. Available: https://www.geeksforgeeks.org/c/discrete-fourier-transform-and-its-inverse-using-c/. [Accessed: Aug. 12, 2026].

[7] Paul R, “How to look up sine of different frequencies from a fixed sized lookup table?,” stackoverflow.com, Nov. 20, 2012. [Online]. Available: https://stackoverflow.com/questions/13466623/. [Accessed: Aug. 12, 2026].

Thank you!

Tool Citations

[8] Apple, GarageBand, ver. 2.3.18, 2025. [Mobile application]. Available: https://apps.apple.com/us/app/garageband/id408709785. [Accessed: Aug. 12, 2026].

[9] LibreOffice, LibreOffice, ver. 26.2.5. [Software]. Available: https://www.libreoffice.org/download/. [Accessed: Aug. 12, 2026].

[10] Freescale, CodeWarrior, ver. 5.9.0, 2008. [Software]. Available: https://moodle.oakland.edu/course/view.php?id=316851. [Accessed: Aug. 12, 2026].

[11] Circuit Diagram, "editor," www.circuit-diagram.org. [Online]. Available: https://www.circuit-diagram.org/editor/. [Accessed: Aug. 12, 2026].

[12] Simon Tatham, PuTTY, ver. 0.84, 2026. [Software]. Available: https://apps.microsoft.com/detail/xpfnzksklbp7rj?hl=en-US&gl=US. [Accessed: Aug. 12, 2026].

Thank you!